

Encryption and the SEXT interface

What libsodium provides, what an application can reach, and why field encryption has no key route in W1.0-0.

SD Core for Windows links libsodium and ships it as `libsodium-26.dll` beside the server. Two things are built on it: the credential exchange that authenticates an API login, and a pair of BASIC functions that encrypt and decrypt a string.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case.

The short version

An administrator reading this page usually wants one of two answers.

For credentials, nothing needs configuring. Passwords are never stored. SD keeps a SCRAM-SHA-256 verifier in the credential store, the API login proves knowledge of the password without sending it, and the primitives behind that exchange are the ones listed further down.

For encrypting application data, W1.0-0 does not provide a usable route. The functions exist and work, but the only way to produce a key they accept is an internal-only call. This is set out in full under *Why field encryption is not available* below, because a site planning to encrypt fields needs to know before it writes the application, not after.

What was removed

The <code>encrypt.field</code> verb	Removed. It pointed at a program, <code>\$CRYPTO</code> , which never existed in the GPL release, so the verb could not have worked in any build derived from it
<code>encrypt()</code> and <code>decrypt()</code>	Removed upstream in July 2024 and replaced by <code>sdencrypt()</code> and <code>sddecrypt()</code> . The old names do not compile
The Python half of SEXT	Removed with the embedded interpreter. The <code>SD_Py*</code> keys, <code>SDPYFUNC.H</code> , twenty <code>PY_*</code> programs and the object opcode all went. A program referencing a Python key does not compile, and the state it tested for cannot be reached

sdencrypt() and sddecrypt()

These are ordinary BASIC functions. Any account may compile a call to them.

```
sdencrypt(data, key, encoding)
sddecrypt(data, key, encoding)
```

Three arguments, not two. The third selects how the key is encoded and how the result is returned:

ENCODING	MEANING	KEY LENGTH REQUIRED
201	hexadecimal	64 characters
202	base64	44 characters

The cipher is libsodium's authenticated `secretbox`, so the key is exactly 256 bits. The two lengths above are that key after encoding, and they are checked exactly: a key of any other length is refused.

WHY FIELD ENCRYPTION IS NOT AVAILABLE

A passphrase is not a key, and this is where a reader will otherwise lose a day. On W1.0-0:

```
sdencrypt('The quick brown fox', 'secretkey', 202)
```

returned nothing and set `status()` to **10204**, a key length error. `secretkey` is nine characters and the function wanted 44.

The function that turns a password into a key of the right length is `sdext()`'s `SD_KEYFROMPW`, and `sdext()` is internal-only — it needs a program compiled with `$internal`, which needs an administrator working in the system account. **So an ordinary program cannot obtain a key these functions will accept, and there is no supported way in.**

An application that must encrypt data in W1.0-0 should do it outside SD, in the client, and store the result as an ordinary string.

The SDEXT interface

`sdext()` is the internal entry point to the cryptographic primitives.

```
rtnval = sdext(arg, isargmv, key)
```

`key` selects the operation, `arg` carries the arguments and `isargmv` says whether `arg` holds several values rather than one.

It cannot be called from an application. `sdxext` is in the compiler's internal function table, so an ordinary account does not merely get refused — it gets a misleading error. An unknown function is read as a matrix reference, so the complaint is about a `dim` statement the program does not contain, reported at the last line rather than at the call. That behaviour is covered by SD Basic - Restricted Commands in this set.

`$internal` needs both halves: the compiler tests for internal mode **and** for the administrator flag. Internal mode alone was enough until 13 August 2026 and was not safe, because internal programs are the only ones that may set the administrator flag.

THE KEYS

Ten are implemented. Every binary value is base64, because the interface carries NUL-terminated strings and a raw 32-byte digest would contain a mark character about one time in nine.

KEY	VALUE	ARGUMENTS	RESULT
<code>SDEXT_TestIt</code>	1	any	Prints each argument and returns a count. A diagnostic
<code>SD_SALT</code>	100	none	A fresh salt, base64
<code>SD_KEYFROMPW</code>	101	password, salt	A 256-bit key derived from the password
<code>SD_EUID_SET</code>	102	user name	Sets the process effective user and group
<code>SD_EUID_RESTORE</code>	103	none	Restores what they were on entry
<code>SD_SHA256</code>	104	one	SHA-256 of the argument
<code>SD_HMACSHA256</code>	105	base64 key, text message	HMAC-SHA256
<code>SD_PBKDF2</code>	106	password, salt, iterations, length	Derived key
<code>SD_RANDBYTES</code>	107	count	Random bytes
<code>SD_XORBYTES</code>	108	two equal-length values	Their exclusive-or
<code>SD_CTEQUAL</code>	109	two values	<code>1</code> or <code>0</code> , compared in constant time

`SD_CTEQUAL` reports a malformed argument as an error rather than as `0`, because by the time the login path compares these values both sides are the server's own — a decode failure there is a defect, not a wrong password. A caller deciding whether to admit a login must still treat the error as a refusal.

`SD_EUID_SET` and `SD_EUID_RESTORE` call the POSIX identity functions provided by the server's runtime. They do not change the Windows process token, and SD's own identity model does not use them.

WHAT USES IT

Eight shipped programs call `sdxext()`, and they are the whole of its use: `APISRV`, `CRED_SET`, `CRED_VERIFY`, `SD_GET_SALT`, `SD_KEY_FROM_PW`, `SDCLIENT`, `EUID_SET` and `EUID_RESTORE`. Between them they set a credential, verify one, and run the API's SCRAM exchange.

None of the eight is catalogued for general use, and none has a VOC entry, so they are not an indirect route to the interface either.

Two constants that are not SDEXT keys

`SD_ENCODEHX` (201) and `SD_ENCODE64` (202) are defined alongside the SDEXT keys and are easily mistaken for them. They are not keys and `sdxext()` does not implement them — passing either as a key returns a key error. They are the third argument to `sdencrypt()` and `sddecrypt()`, and they are documented above.
